Mathematical contract and exact acceptance derivation

Scope and provenance

Hypra is a **partial native Python/SymPy implementation** for exact regulator expansions. This chapter gives detailed proofs for three illustrative families and the P/Q/J4 acceptance calculation, followed by the general integer and half-integer kernels. These examples are not the package's full supported scope; consult the capability map for current coverage and limits. The native backend is the default and **requires no Mathematica installation or setup**. Unsupported cases must fail explicitly rather than be presented as complete expansions.

HypExp [1] expands generalized hypergeometric functions around integer-valued parameters. HypExp 2 [2] adds certain half-integer classes. Hypra does not yet match their full coverage. An optional external HypExp adapter is retained as an experimental, **unverified** alternative; it is neither required by the native implementation nor evidence that a real Wolfram/HypExp run succeeded.

The calculations below independently derive the requested example. Earlier private project material was context only: no private code or integral lookup table is reproduced or imported here. The reproducible evidence is Hypra's public Python validation source, not a private file reference.

Public implementation excerpt

`derive_moments()`, lines 20–45 derives the five logarithmic moments using convergent sums, beta-function coefficients and a dilogarithm reflection identity. The excerpt below is copied verbatim apart from indentation from lines 26–38 of the pinned published revision `4b2904a`. Here `s` is SymPy, and `a` and `t` are formal expansion variables.

```
n = s.Symbol("n", integer=True, positive=True)
a, t = s.symbols("a t")
log_gamma_ratio = s.expand(
    sum((-1) ** k * s.zeta(k) * (a**k + t**k - (a + t) ** k) / k for k in range(2, 5))
)
# Beta(a, 1+t) = exp(log_gamma_ratio)/a. Taking d_a d_t ^2
# means twice the a^2*t^2 coefficient before division by a.
beta_mixed = s.expand(log_gamma_ratio + log_gamma_ratio**2 / 2)
b = s.simplify(2 * beta_mixed.coeff(a, 2).coeff(t, 2))
zeta2_from_series = s.summation(n**-2, (n, 1, s.oo))
# Li2(1-w) = zeta(2)-Li2(w)-log(w)log(1-w), and
# integral Li2(w)log(1-w) dw/w = -Li2(1)^2/2.
c = s.simplify(-(zeta2_from_series**2) / 2 - b)
```

The mixed coefficient `b` gives $\int_0^1 \log(w)\log^2(1-w)\,dw/w$; `c` is the moment containing $\log(1-w)\,\mathrm{Li}_2(1-w)$. The mathematical justification for extracting these coefficients is developed below. The subsequent returned moment dictionary uses exact convergent zeta sums, and `derive_j4()` combines the resulting moments into the quadratic and cubic regulator coefficients. No fitted or hard-coded J4 coefficient is inserted. This Python validation does not claim that an external Mathematica/HypExp installation was run.

Conventions

- The expansion variable is a dimensionless regulator ϵ, expanded at zero.

- `through=k` means every Laurent term through and **including** ϵ^k, plus a remainder $O(\epsilon^{k+1})$. Pole terms must not be discarded.
- Inputs and coefficients are exact. A binary floating-point approximation is not a substitute for a rational parameter.
- The derivation in this document uses **real** $0 < w < 1$, principal real logarithms and classical polylogarithms. Continuation outside this interval requires a branch prescription; the formulas must not be silently reused across cuts.
- An HPL object retains its complete index word and argument. Under the usual real alphabet convention, $f_0(t) = 1/t$, $f_1(t) = 1/(1-t)$ and $f_{-1}(t) = 1/(1+t)$, with $H_0(z) = \log z$, $H_1(z) = -\log(1-z)$, and $H_{-1}(z) = \log(1+z)$. Compact nonzero index m denotes $|m|-1$ zeros followed by $\mathrm{sgn}(m)$, not a different kernel. Complex continuation, extended alphabets, and endpoint regularization are separate matters; a wrapper must not invent their conventions.

Native all-order HPL recursion for an integer family

The native implementation is based on a differential equation, not stored acceptance coefficients. Let

$$F(z,\epsilon) = {}_2F_1(1, b\epsilon; 1+c\epsilon; z) = 1 + \sum_{k\geq 1} \epsilon^k f_k(z),$$

where b, c are regulator-independent exact constants. Its argument-series coefficient $a_n = (b\epsilon)_n/(1+c\epsilon)_n$ satisfies $(n+c\epsilon)a_n = (n-1+b\epsilon)a_{n-1}$ for $n \geq 1$. Summing this identity gives

$$z(1-z)\partial_z F + \epsilon(c-bz)F = c\epsilon, \qquad F(0,\epsilon) = 1.$$

At order one $f_1' = b/(1-z)$, hence $f_1 = bH_1(z)$. At every higher order,

$$f_k'(z) = \left[-\frac{c}{z} + \frac{b-c}{1-z}\right] f_{k-1}(z), \qquad f_k(0) = 0.$$

With $\mathcal{I}_a$ denoting the operation of prepending letter a to an HPL word, this is the constructive recurrence

$$\boxed{f_1 = bH_1, \qquad f_k = -c\,\mathcal{I}_0 f_{k-1} + (b-c)\,\mathcal{I}_1 f_{k-1} \quad (k \geq 2).}$$

Every word ends in 1, so its origin value is zero and these integrals require no divergent origin regularization. Equivalently, a word of length k ending in 1, with n_0 zero letters and n_1 one letters, has coefficient $b(-c)^{n_0}(b-c)^{n_1-1}$. This proves every order by induction and uniqueness of the regular solution, not only the first two acceptance orders. It covers P with $(b,c) = (-1,-2)$ and Q with $(b,c) = (-2,-2)$ and argument $1-w$.

The derivation starts on $|z| < 1$ near $\epsilon = 0$. For real $0 < z < 1$ it uses the unambiguous real HPL branch. Analytic continuation follows only with matching HPL path/branch conventions; no branch cut crossing is implicit. The offline recurrence tests compare orders 1 through 5, at argument powers 1 through 6, against exact Pochhammer quotients for four choices of (b,c). The proof, not that finite test sample, establishes the all-order recurrence.

Two narrowly supported half-integer families

These identities do **not** establish general half-integer HypExp 2 coverage. Let $t = \sqrt{z}$ with real $0 < z < 1$.

Even-binomial family

For a independent of the regulator,

$$G(z,\epsilon) = {}_2F_1(a\epsilon, \tfrac{1}{2} + a\epsilon; \tfrac{1}{2}; z) = \frac{(1-t)^{-2a\epsilon} + (1+t)^{-2a\epsilon}}{2}.$$

To prove it, take the even terms of the binomial series and use $(2A)_{2n} = 2^{2n}(A)_n(A+1/2)_n$ and $(2n)! = 2^{2n}n!(1/2)_n$, with $A = a\epsilon$. Thus

$$[\epsilon^k]G = \frac{(-2a)^k}{2k!}\left[\log^k(1-t) + \log^k(1+t)\right], \quad k \geq 0.$$

The expression is even in t and has a regular $z = 0$ limit. Its derivation from the binomial identity is exact, not a fitted expansion. No cancellation of upper and lower hypergeometric parameters at nonzero regulator was used.

Half-integer telescoping family

For $A = 1/2 + b\epsilon$,

$$T(z,\epsilon) = {}_2F_1(1, A; 1+A; z) = \sum_{n\geq 0} \frac{A}{n+A} z^n.$$

Define

$$S_k(z) = \sum_{n\geq 0} \frac{z^n}{(n+1/2)^k} = \frac{2^{k-1}}{t}\left[\mathrm{Li}_k(t) - \mathrm{Li}_k(-t)\right].$$

The equality follows by extracting odd terms of the polylogarithm series. Since $A/(n+A) = 1 - n/(n+A)$, direct geometric expansion of its denominator gives

$$T_0 = \frac{\log(1+t) - \log(1-t)}{2t}, \qquad [\epsilon^k]T = (-1)^{k-1}b^k\left[S_k - \frac{1}{2}S_{k+1}\right] \quad (k \geq 1).$$

This includes the $n = 0$ term correctly: it contributes 1 at order zero and zero at every higher order. For each fixed finite b, the expansion is local in ϵ, before the nearest denominator pole. At $z = 0$ the displayed quotients have removable limits ($T_0 = 1$, higher coefficients zero), whereas $z = 1$ is divergent. An implementation must handle those endpoints explicitly, not evaluate a spurious $0/0$ or silently return a finite value at 1.

P and Q from their defining series

Set

$$P(w,\epsilon) = {}_2F_1(1,-\epsilon;1-2\epsilon;w), \qquad Q(w,\epsilon) = {}_2F_1(1,-2\epsilon;1-2\epsilon;1-w).$$

The upper parameter -2ϵ in Q is **not** the lower parameter $1-2\epsilon$; no upper/lower cancellation applies.

For $n \geq 1$, using $(1)_n/n! = 1$,

$$\frac{(-\epsilon)_n}{(1-2\epsilon)_n} = -\frac{\epsilon}{n}\left[1 + \epsilon\left(H_{n-1} + \frac{2}{n}\right) + O(\epsilon^2)\right].$$

This follows by writing the rising factorials as products and differentiating $\sum_{j=1}^{n-1}\log(1-\epsilon/j) - \sum_{j=1}^{n}\log(1-2\epsilon/j)$. On $|w|<1$,

$$\sum_{n\geq 1}\frac{w^n}{n} = -\log(1-w), \quad \sum_{n\geq 1}\frac{w^n}{n^2} = \mathrm{Li}_2(w), \quad \sum_{n\geq 1}\frac{H_{n-1}w^n}{n} = \frac{1}{2}\log^2(1-w).$$

The last identity follows by differentiating both sides, using $\sum_{n\geq 1} H_{n-1}w^{n-1} = -\log(1-w)/(1-w)$, and the value zero at $w=0$. Consequently

$$P = 1 + \epsilon\log(1-w) + \epsilon^2\left[-2\,\mathrm{Li}_2(w) - \frac{1}{2}\log^2(1-w)\right] + O(\epsilon^3).$$

For Q there is an exact telescoping coefficient:

$$\frac{(-2\epsilon)_n}{(1-2\epsilon)_n} = -\frac{2\epsilon}{n-2\epsilon}. \qquad Q = 1 - \sum_{j\geq 1}(2\epsilon)^j\,\mathrm{Li}_j(1-w).$$

The double-series interchange is justified for fixed $0<w<1$ and $|2\epsilon|<1$ by absolute convergence. Hence

$$Q = 1 + 2\epsilon\log w - 4\epsilon^2\,\mathrm{Li}_2(1-w) + O(\epsilon^3).$$

The finite-n coefficient identities are checked exactly by the offline tests; they are not numerically fitted.

Why the J4 expansion may be integrated

Define

$$J_4(\epsilon) = \int_0^1 w^{-1-2\epsilon}(1-w)^{-\epsilon}(P-1)(Q-1)\,dw.$$

Fix a sufficiently small closed complex disk $|\epsilon| \le r < 1/4$. All hypergeometric denominator parameters avoid their singular values there. The exact Q coefficients imply $|Q-1| \le C_r|\epsilon||\log w|$ near $w=0$ and $|Q-1| \le C_r|\epsilon|(1-w)$ near $w=1$. For P, its coefficient can be written

$$-\epsilon \frac{\Gamma(1-2\epsilon)}{\Gamma(1-\epsilon)} \frac{\Gamma(n-\epsilon)}{\Gamma(n+1-2\epsilon)}.$$

Uniform gamma-ratio asymptotics on this compact regulator disk give $|p_n| \le C_r|\epsilon|n^{r-1}$. Therefore $P-1 = O_r(|\epsilon|w)$ near zero and $P-1 = O_r(|\epsilon|(1-w)^{-r})$ near one. The full integrand is bounded by constants times

$$w^{-2r}|\log w| \quad (w \to 0), \qquad (1-w)^{1-2r} \quad (w \to 1),$$

both integrable. Dominated holomorphic parameter integration and Cauchy's formula on a smaller disk permit differentiation under this integral to any fixed finite order. Thus the Taylor coefficients computed below are legitimate integrals; the integrated remainder is $O(\epsilon^4)$. This proof is specific to J_4, **not** permission to expand each unsubtracted singular term in a larger NNLO integral separately.

Deriving all needed moments, not importing a table

All integrals in this subsection have measure dw/w on $(0,1)$. Write $L = \log w$, $M = \log(1-w)$ and $\zeta_k = \zeta(k)$. Expanding the logarithm or dilogarithm and integrating their absolutely integrable terms gives

$$\int LM = \sum_{n\ge1} \frac{1}{n^3} = \zeta_3, \qquad \int L\,\mathrm{Li}_2(w) = -\sum_{n\ge1} \frac{1}{n^4} = -\zeta_4,$$

$$\int L^2 M = -2\sum_{n\ge1} \frac{1}{n^4} = -2\zeta_4.$$

Here $\int_0^1 w^{n-1}\log w\,dw = -1/n^2$ and $\int_0^1 w^{n-1}\log^2 w\,dw = 2/n^3$. The absolute sums converge.

For $B = \int LM^2$, use $\mathrm{B}(a,1+t) - 1/a$, analytic at $(a,t) = (0,0)$, and take $\partial_a\partial_t^2$. The relevant generating function is

$$\mathrm{B}(a,1+t) = \frac{1}{a}\exp\left\{\sum_{k\ge2} \frac{(-1)^k\zeta_k}{k}[a^k + t^k - (a+t)^k]\right\}.$$

The a^2t^2 coefficient of the exponential is $-3\zeta_4/2 + \zeta_2^2/2 = -\zeta_4/4$. After division by a and the two t derivatives, $B = -\zeta_4/2$. Differentiation is justified either after subtracting $1/a$ or directly for the mixed derivative, whose logarithmic integrand is integrable.

Finally, with $C = \int M\,\mathrm{Li}_2(1-w)$, the real reflection identity $\mathrm{Li}_2(w) + \mathrm{Li}_2(1-w) = \zeta_2 - LM$ gives

$$C = -\zeta_2^2 + \frac{1}{2}\zeta_2^2 - B = -\frac{1}{2}\zeta_2^2 - B = -\frac{3}{4}\zeta_4.$$

We used $d\,\mathrm{Li}_2(w) = -M\,dw/w$ and its endpoint values; all boundary terms are finite.

Assemble the exact coefficients

Expand the weight as $w^{-1}[1+\epsilon(-2L-M)+O(\epsilon^2)]$. Multiplying the derived series gives

$$[\epsilon^2]J_4 = \int 2LM = 2\zeta_3,$$

$$[\epsilon^3]J_4 = \int [-4L\,\mathrm{Li}_2(w) - 3LM^2 - 4M\,\mathrm{Li}_2(1-w) - 4L^2M] = \left(4+\frac{3}{2}+3+8\right)\zeta_4 = \frac{33}{2}\zeta_4.$$

Thus the requested result is consistent:

$$\boxed{J_4 = 2\zeta(3)\epsilon^2 + \frac{11\pi^4}{60}\epsilon^3 + O(\epsilon^4).}$$

The script creates the polynomial product and integrates each monomial using the independently derived moments. It rejects any unproved monomial rather than supplying a guessed value.

Laurent prefactors and order budgeting

If a prefactor A starts at ϵ^{-p}, an analytic factor F is generally needed through order $k+p$ to obtain AF through order k. For example, $\Gamma(\epsilon)^2F$ needs F through ϵ^{k+2}, not merely ϵ^k. Multiplication, exact cancellations and the final truncation must be performed on the **whole expression**. If a backend cannot establish the valuation/order budget or its result still contains an unexpanded hypergeometric function, a regulator derivative or a regulator-dependent coefficient, it must fail rather than label the result complete. Assumptions needed to establish a valuation or cancel a parameter are part of the mathematical input.

Validation and limits

```
uv run python scripts/validate_integral.py
uv run python scripts/validate_integral.py --latex
uv run python scripts/validate_integral.py --numeric
uv run pytest tests/test_mathematics.py
```

The optional numerical check evaluates the original integral using contiguous representations for $P-1$ and $Q-1$ to avoid catastrophic subtraction near the endpoints. It uses negative regulator samples, for which endpoint limits are zero, and introduces **no endpoint cutoff**. Convergence of the residual scaled by ϵ^4 is secondary support only, not the analytic proof and not a real-backend validation. No global continuation or arbitrary-HPL simplifier is established by these examples.

References

1. T. Huber and D. Maître, *HypExp, a Mathematica package for expanding hypergeometric functions around integer-valued parameters*, arXiv:hep-ph/0507094.
2. T. Huber and D. Maître, *HypExp 2, Expanding Hypergeometric Functions about Half-Integer Parameters*, arXiv:0708.2443.

The abstract pages were checked for these scope statements. No source code from these packages is redistributed with this derivation.

Native general-parameter kernels

The implementation now constructs canonical integer coefficients by strict nested-sum algebra and their HPL generating functions. Matched half-integer parameters use even-index projections and colored strict sums. Parameter shifts act with the hypergeometric differential operator

$$L = \theta \prod_j (\theta + b_j - 1) - z \prod_i (\theta + a_i), \qquad \theta = z\frac{d}{dz}.$$

For example, raising an upper parameter applies $F(a_i + 1) = (\theta + a_i)F/a_i$, and lowering a lower parameter applies $F(b_j - 1) = (\theta + b_j - 1)F/(b_j - 1)$. The inverse shifts follow polynomial division of the same operator. Each zero-base divisor increases the canonical regulator depth before any truncation. Apparent Laurent poles must cancel exactly below the proven pole bound. An identically zero shift divisor is resolved by a single formal slope perturbation only when its finite limit can be established algebraically.

For all-half bases at rank p,

$$L_0 = (\theta - \tfrac{1}{2})^{p-1} B, \qquad B = (1 - z)\theta - \tfrac{z}{2}.$$

The order of these operators is essential. Set $F = (1-z)^{-1/2}Y$, $t = i\sqrt{z/(1-z)}$, so $z = t^2/(t^2-1)$. The gauge-conjugated operators are

$$\tilde{\theta} = \frac{t(1-t^2)}{2}\partial_t - \frac{t^2}{2}, \quad \widetilde{D} = \frac{t(1-t^2)}{2}\partial_t - \frac{1+t^2}{2}, \quad \widetilde{B} = \frac{t}{2}\partial_t.$$

Their origin-normalized inverses are

$$\widetilde{D}^{-1} g(t) = \frac{t}{1-t^2} \int_0^t \frac{2g(u)}{u^2}\, du, \qquad \widetilde{B}^{-1} g(t) = \int_0^t \frac{2g(u)}{u}\, du.$$

At each regulator order apply $\widetilde{D}^{-1}$ $p-1$ times and then $\widetilde{B}^{-1}$. The initial coefficient is $Y_0 = 1$; correction sources and solutions are $O(t^2)$, making these integrals convergent. The odd homogeneous solution of $\widetilde{D}$ is excluded by analyticity in z, and the constant homogeneous solution of $\widetilde{B}$ is fixed by $F(0) = 1$. Only HPL letters $0, \pm 1$ are required. For real $0 < z < 1$, continuation follows the positive imaginary t axis with principal square roots. This does not assert arbitrary complex-argument continuation or permission to interchange integrals and limits.

These are independent Python-native mathematical implementations inspired by the two cited HypExp papers. The detailed capability map and resource restrictions are in `scope.md`; canonical class coverage is not a claim of complete HypExp/HPL API parity.